

Cours : Installation de PHP sur Windows

I. Introduction

PHP est un langage côté serveur utilisé pour créer des applications web dynamiques. Avant de commencer à programmer en PHP, il faut l'installer sur sa machine.

Dans ce cours, nous allons voir :

- Installer PHP sur **Windows**

- Tester si l'installation fonctionne

- Configurer PHP dans le terminal

- Installer un serveur local (Apache / Nginx / Laravel Valet selon l'OS)

II. Installer PHP sur Windows

- Méthode 1 : Installer XAMPP (plus simple pour les étudiants)

XAMPP = Apache + PHP + MySQL + phpMyAdmin

Étapes

1. Télécharger XAMPP

Aller sur : <https://www.apachefriends.org/fr/index.html>

Choisir **XAMPP pour Windows**

Télécharger la version contenant PHP 8.x

2. Installer XAMPP

Double-clique sur le fichier .exe

Laisse les options par défaut :

- Apache
- MySQL
- PHP
- phpMyAdmin

3. Lancer Apache

Ouvre **XAMPP Control Panel**

Clique sur **Start** devant Apache

Clique sur **Start** devant MySQL (si besoin)

4. Vérifier PHP:

Ouvre ton navigateur

Tape :http://localhost

Dans C:\xampp\htdocs, crée un fichier :test.php

Avec :

```
<?phpphpinfo();
```

—> Ouvre <http://localhost/test.php>

Si tu vois la page phpinfo = PHP est installé

Méthode 2 : Installer PHP manuellement (professionnel)

1. Télécharger PHP

Va sur : <https://windows.php.net/download/>

Télécharge :

- **VC15 x64 Thread Safe** (ou la version récente)

2. Extraire le dossier:

Dézippe dans :C:\php

3. Ajouter PHP dans les variables d'environnement

- Cherche : **Modifier les variables d'environnement**
- Clique sur **Path**

Ajoute :C:\php

4. Vérifier l'installation

Ouvrir **PowerShell** ou **CMD** :

```
php -v
```

Si tu obtiens la version → PHP installé

III. Installer PHP sur macOS

Méthode 1 : Installation avec Homebrew (la meilleure)

Étape 1 : Installer Homebrew

Dans le Terminal :

```
/bin/bash -c "$(curl -fsSL  
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Ensuite :

```
brew --version
```

Étape 2 : Installer PHP

```
brew install php
```

Étape 3 : Démarrer PHP

```
php -v
```

Étape 4 : Démarrer le serveur interne de PHP

Dans un dossier :

```
php -S localhost:8000
```

PHP fonctionne !

Méthode 2 : Installer MAMP (simple pour les débutants)

1. Télécharger

Va sur : <https://www.mamp.info/en/downloads/>

2. Installer

Glisser dans **Applications**

3. Lancer MAMP

Click sur **Start Servers**

4. Test PHP

Créer dans :

/Applications/MAMP/htdocs/test.php

Avec :

```
<?php phpinfo(); ?>
```

Ouvrir :

<http://localhost:8888/test.php>

→ PHP OK

IV. Tester si PHP est bien installé

Dans Windows ou macOS :

```
php -v
```

Résultat attendu :

PHP 8.x.x (cli) ...

V. Configurer Visual Studio Code pour PHP

1. Installer VS Code

Installer l'extension **PHP Intelephense**

Créer un fichier .php et vérifier la coloration syntaxique.

VI. Installer Composer (gestionnaire de packages PHP)

Windows :

Télécharger :

<https://getcomposer.org/Composer-Setup.exe>

macOS :

```
brew install composer
```

Vérifier :

composer -v

Chapitre 0:AFFICHER UN MESSAGE À L'ÉCRAN

En PHP, pour afficher quelque chose dans la console ou dans une page, on utilise :

echo ou print

Exemple :

```
<?php echo "=== BIENVENUE DANS LE COURS COMPLET DE PHP ===";
```

Retour à la ligne

En PHP, on utilise `\n` pour aller à la ligne dans la console.

```
echo "Bienvenue dans le cours complet de PHP\n";
```

Pour exécuter ton fichier PHP

Tu écris ton fichier dans VS Code puis tu exécutes :

```
php mon_fichier.php
```

Chapitre 1 : LES COMMENTAIRES EN PHP

Comme en Dart, PHP possède 2 styles de commentaires :

Commentaire sur une ligne

// Ceci est un commentaire sur une ligne

Exemple :

```
<?php //echo "=== BIENVENUE DANS LE COURS COMPLET DE PHP ==="; ?>
```

Commentaire sur plusieurs lignes

```
/*  
    Ceci est un commentaire  
    sur plusieurs lignes  
*/
```

Exemple :

```
<?php  
/*echo "=== BIENVENUE DANS LE COURS COMPLET DE PHP ===";  
  
echo "=== BIENVENUE DANS LE COURS COMPLET DE PHP ===";  
*/  
  
?>
```

Chapitre 2 : VARIABLES ET CONSTANTES

Une variable sert à **stocker une valeur**.

En PHP, une variable commence toujours par \$.

Exemple :

```
<?php $age = 25;
```

Signification :

\$age → nom de variable

= → affectation

25 → valeur

A- Les types en PHP (simple)

PHP est un langage **faiblement typé**.

Tu n'as pas besoin d'indiquer les types (comme int, String, bool en Dart).

Type	Exemple
integer (entier)	25
float (décimal)	1.82
string (texte)	"Salut"
bool	true / false

B-Constantes

const et define

```
const PAYS = "Sénégal";define("VILLE", "Dakar");
```

Exercice (version PHP)

Créer :

- une variable \$taille (float)
- une variable \$estDeveloppeur (bool)
- une constante PAYS
- une constante VILLE
- une variable \$nom avec ton prénom

Solution :

```
<?php  
$taille = 1.82;  
$estDeveloppeur = true;
```

```
const PAYS = "Sénégal";
define("VILLE", "Dakar");
$nom = "Ciko";
```

C-Afficher une variable

➤ Méthode classique

```
echo "Nom : " . $nom;
```

➤ Méthode moderne (interpolation)

PHP utilise **{variable}** :

```
echo "Nom : {$nom}\n";
```

D-LIRE UNE SAISIE UTILISATEUR

En PHP (console), on utilise **readline()**.

Exemple :

```
<?php $prenom = readline("Entrez votre prénom : ");
echo "Salut $prenom, ravi de te rencontrer !\n";
```

E-CHAÎNES DE CARACTÈRES (STRING)

Déclarer une chaîne :

```
$phrase = "Hello PHP!";
```

Méthodes utiles

Méthode	PHP
length	strlen(\$phrase)
majuscule	strtoupper(\$phrase)
minuscule	strtolower(\$phrase)
remplacer	str_replace("PHP", "World", \$phrase)

Exemple :

```
$phrase = "Hello PHP!";
```

```
echo "Longueur : " . strlen($phrase) . "\n";echo "Majuscules : " . strtoupper($phrase) .
"\n";echo "Minuscules : " . strtolower($phrase) . "\n";echo "Remplacer : " .
str_replace("PHP", "World", $phrase) . "\n";
```

F-CONVERSION DE TYPES

String → int

```
$nombreTexte = "123";$enInt = intval($nombreTexte);
```

String → float

```
$enDouble = floatval("3.14");
```

Nombre → string

```
$texte = strval($enInt);
```

G-OPÉRATEURS ARITHMÉTIQUES

Mauvaise pratique :

```
$a = 10;$b = 3;echo $a + $b;
```

Bonne pratique :

```
$nombrePrincipal = 10;$nombreSecondaire = 3;
echo "Addition : " . ($nombrePrincipal + $nombreSecondaire) . "\n";echo
"Soustraction : " . ($nombrePrincipal - $nombreSecondaire) . "\n";echo
"Multiplication : " . ($nombrePrincipal * $nombreSecondaire) . "\n";echo "Division :
" . ($nombrePrincipal / $nombreSecondaire) . "\n";echo "Modulo : " .
($nombrePrincipal % $nombreSecondaire) . "\n";
```

Chapitre 3: Structure de contrôle

Il existe **2 grandes familles** de structures de contrôle : **structure conditionnelle et Boucle**

I-STRUCTURES CONDITIONNELLES:

Une structure conditionnelle est une instruction qui permet à un programme de prendre une décision. elle exécute un bloc de code si une condition est vraie et un autre si elle est fausse

Condition simple

```
$estDeveloppeur = true;
if ($estDeveloppeur) {
    echo "Affichage du contenu complet pour développeur.\n";
} else {
    echo "Affichage du contenu restreint.\n";
}
```

Condition avec nombre

```
$age = 20;
if ($age > 18) {
    echo "Accès complet autorisé.\n";
} else {
    echo "Accès limité.\n";
}
```

II- Les Boucles :

Les **boucles** permettent de répéter une instruction plusieurs fois sans réécrire le même code.

Elles sont indispensables en programmation : afficher une liste, parcourir des données, compter, etc.

En PHP, on utilise principalement **4 types de boucles** :

- **While**
- **do...while**
- **For**
- **foreach**

1-La boucle while:

La boucle while répète du code **tant qu'une condition est vraie**.

Structure :

```
while (condition) {  
    // instructions répétées  
}
```

Exemple :

```
$i = 1;  
while ($i <= 5) {  
    echo "Nombre : $i <br>";  
    $i++;  
}
```

Explication :

On commence à 1

La boucle tourne tant que $i \leq 5$

On incrémente i à chaque tour.

Résultat à l'écran :

Nombre : 1Nombre : 2Nombre : 3Nombre : 4Nombre : 5

2-La boucle do...while

C'est comme while, **mais la boucle s'exécute au moins une fois**, même si la condition est fausse.

Structure :

```
do {  
    // instructions  
} while (condition);
```

Exemple :

```
$i = 1;  
do {  
    echo "Tour : $i <br>";  
    $i++;  
}
```

```
} while ($i <= 3);
```

3-La boucle for

Utilisée quand on sait **à l'avance** combien de fois on veut répéter le code.

Structure :

```
for (initialisation; condition; incrémentation) {  
    // instructions  
}
```

Exemple :

```
for ($i = 1; $i <= 5; $i++) {  
    echo "Valeur : $i <br>";  
}
```

4- La boucle foreach

Spécialement utilisée pour **parcourir les tableaux**.

Structure :

```
foreach ($tableau as $valeur) {  
    // instructions  
}
```

Exemple :

```
$prenoms = ["Chrislain", "Amina", "Fatou"];  
foreach ($prenoms as $p) {  
    echo "Bonjour $p <br>";  
}
```

Chapitre 4 :COLLECTIONS : LISTES, SETS, MAPS (version PHP)

PHP ne possède pas List / Set / Map comme en Java.
Mais on utilise **les tableaux (arrays)**.

1. LISTE (array indexé):

C'est un conteneur ordonné qui stocke plusieurs valeurs sous un seul nom. Chaque élément a une position numérique (index) qui commence généralement à 0.

```
<?php

// 1. LISTE (array indexé)
$nombre = [1, 2, 3, 4, 5];

// Afficher le premier élément (index 0)
echo $nombre[0]; // 1

// Ajouter un élément à la fin du tableau
$nombre[] = 6; // ajouter un élément

// Supprimer l'élément à l'index 2 (valeur 3)
unset($nombre[2]); // supprimer l'élément 3

// Afficher le tableau
print_r($nombre);

?>
```

2. SET (ensemble sans doublon)

PHP n'a pas Set, mais tu peux simuler :

```
<?php

// 1. Créer un tableau avec un doublon
$pays = ["USA", "Canada", "France", "USA"];
// État: ["USA", "Canada", "France", "USA"] (2x USA)

// 2. Transformer en Set (supprimer les doublons)
$pays = array_unique($pays);
// État: [0=>"USA", 1=>"Canada", 2=>"France"]

// 3. Ajouter un nouveau pays
$pays[] = "Japon";
// État: [0=>"USA", 1=>"Canada", 2=>"France", 3=>"Japon"]
```

```
// 4. Supprimer un pays (Canada)
unset($pays[1]);
// État: [0=>"USA", 2=>"France", 3=>"Japon"]
// Note: l'index 1 manque maintenant!
```

```
// 5. Afficher
print_r($pays);
```

```
?>
```

2. MAP (tableau associatif):

Un Map (ou tableau associatif) est comme un dictionnaire :

Clé → Valeur

Mot → Définition

Pays → Capitale

Différence avec une liste normale

Liste normale (indexée)

```
<?php
```

```
// Index numérique (0, 1, 2...) $liste = ["Paris", "Lyon", "Marseille"]; echo $liste[0]; //
"Paris" // Index: 0, 1, 2... toujours des nombres
```

```
?>
```

Map (associatif)

```
<?php
```

```
// Clé personnalisée (texte ou autre) $map = ["France" => "Paris", "USA" => "Washin
gton"]; echo $map["France"]; // "Paris" // Clé: "France", "USA" → plus lisible !
```

```
?>
```


Analogies concrètes

1. Comme un annuaire téléphonique

```
<? php
```

```
$annuaire = [
```

```
    "Alice" => "0123456789",
```

```
    "Bob" => "0987654321",
```

```
    "Charlie" => "0666666666"];echo $annuaire["Alice"]; // "0123456789"
```

```
?>
```

2. Comme une fiche d'identité

```
<?php
```

```
$personne = [
```

```
    "nom" => "Dupont",
```

```
    "age" => 30,
```

```
    "ville" => "Paris",
```

```
    "metier" => "Développeur"];echo $personne["nom"]; // "Dupont"echo $personne  
["age"]; // 30
```

```
?>
```

3. Comme un menu de restaurant:

```
<?php
```

```
$menu = [
```

```
    "pizza" => 12.50,
```

```
    "pasta" => 9.99,
```

```
"salad" => 7.50,
```

```
"soda" => 3.00];echo "Prix de la pizza : " . $menu["pizza"] . "€";
```

```
?>
```

CHAPITRE 5 : LES FONCTIONS EN PHP

Après avoir vu les collections, il est temps de découvrir un concept essentiel en programmation : **les fonctions**.

Une fonction est :

- un bloc de code réutilisable
- qui exécute une tâche précise
- qui peut recevoir des paramètres
- et peut retourner une valeur

1- Fonction simple sans retour

En PHP, une fonction se définit avec le mot clé **function** :

Déclaration d'une fonction simple

```
<?php
function afficherMessage() {
    echo " Bienvenue dans le monde des fonctions en PHP !\n";
}
```

Appel de la fonction

```
afficherMessage();
```

Bonnes pratiques (équivalent des dossiers lib/bin de Dart)

En Dart :

- lib = fonctions réutilisables
- bin = point d'entrée

En PHP, on fait exactement la même chose avec des **fichiers organisés**.

Exemple d'organisation :

```
mon_projet/
├── fonctions.php (contient les fonctions)
└── main.php      (point d'entrée)
```

fonctions.php

```
<?php
function afficherMessage() {
    echo " Bienvenue dans le monde des fonctions en PHP !\n";
}
```

main.php

```
<?php
include "fonctions.php";
afficherMessage();
```

Avantages :

- **Lisibilité** : toutes les fonctions regroupées dans un fichier.
- **Clarté** : un fichier principal pour exécuter le programme.
- **Réutilisabilité** : tu peux utiliser fonctions.php dans plusieurs projets.
- **Évolutivité** : tu ajoutes des fonctions sans casser le reste.

2- Fonction avec paramètres

Comme en Dart, une fonction peut recevoir des valeurs.

Exemple en PHP :

```
<?php
function afficherNombre($nombre) {
    echo " Mon nombre est : $nombre\n";
}
afficherNombre(42);afficherNombre(100);
```

3- Fonction qui retourne une valeur

En PHP, on utilise aussi **return**.

```
<?php
function creerTexteNombre($nombre) {
    return " Mon nombre est $nombre";
}
$texte = creerTexteNombre(25);echo $texte . "\n";
echo creerTexteNombre(50) . "\n";
```

4- Fonction avec plusieurs paramètres

```
function additionner($a, $b) {  
    return $a + $b;  
}  
$resultat = additionner(10, 20);echo "    La somme de 10 et 20 est : $resultat\n";
```

Bonnes pratiques : choisir de bons noms

En Dart comme en PHP, des variables comme \$a et \$b ne sont **pas** explicites.

Mauvaise pratique :

```
function additionner($a, $b) {  
    return $a + $b;  
}
```

On ne sait pas ce que représentent \$a et \$b.

Bonne pratique : noms explicites

```
function additionner($premierNombre, $deuxiemeNombre) {  
    return $premierNombre + $deuxiemeNombre;  
}  
$resultat = additionner(10, 20);echo "    La somme de 10 et 20 est : $resultat\n";
```

Autre exemple : contexte financier

```
function additionnerMontants($montant1, $montant2) {  
    return $montant1 + $montant2;  
}  
$total = additionnerMontants(1500.50, 2499.99);echo " Le total des montants est :  
$total FCFA\n";
```

5- Paramètres nommés ?

PHP **n'a pas** officiellement de paramètres nommés comme Dart.
Mais depuis PHP 8, on peut simuler cette fonctionnalité lors de l'appel :

```
function creerProfil($nom, $age, $pays) {  
    echo " $nom, $age ans, vient de $pays\n";  
}  
creerProfil(  
    pays: "Sénégal",  
    age: 25,  
    nom: "Ciko"
```

);

6- Paramètres optionnels

PHP permet de donner une valeur par défaut :

```
function saluer($nom = "invité") {  
    echo "Bonjour, $nom !\n";  
}  
saluer(); // Bonjour, invité !saluer("Chris"); // Bonjour, Chris !
```

7-Fonctions fléchées (=>)

PHP possède aussi des fonctions fléchées (comme en Dart) :

```
$carre = fn($x) => $x * $x;  
echo $carre(5); // 25
```

8- Fonction anonyme

PHP accepte aussi les fonctions anonymes :

```
$bonjour = function($nom) {  
    return "Bonjour $nom !";  
};  
echo $bonjour("Chris");
```

9- Fonction passée en paramètre (callback)

```
function executerOperation($operation, $a, $b) {  
    echo "Résultat : " . $operation($a, $b) . "\n";  
}  
$addition = fn($x, $y) => $x + $y;  
executerOperation($addition, 5, 10);
```

Fonction asynchrone ?

PHP n'est pas nativement asynchrone comme Dart.
Mais on peut simuler un délai :

```
function attendre($secondes) {  
    echo "    Attente de $secondes secondes...\n";  
    sleep($secondes);  
}
```

```
    echo "    Terminé\n";  
}  
attendre(2);
```

Fonction principale (équivalent du main de Dart)

```
<?php  
include "fonctions.php";  
echo " Démonstration des fonctions en PHP\n\n";  
afficherMessage();echo "-----\n";  
afficherNombre(42);echo "-----\n";  
echo creerTexteNombre(25) . "\n";echo "-----\n";  
echo additionner(10, 20) . "\n";echo "-----\n";  
attendre(2);echo "-----\n";  
echo " Fin du cours sur les fonctions en PHP !\n";  
?>
```

CHAPITRE 6: La Programmation Orientée Objet (POO) en PHP

Après avoir exploré les bases du langage PHP, il est temps de structurer le code d'une manière plus professionnelle : **la Programmation Orientée Objet (POO)**.

La POO permet d'organiser le code autour d'objets inspirés du monde réel, créés à partir de modèles appelés **classes**.

Dans ce chapitre, tu vas apprendre :

- Les classes et objets
- Les constructeurs
- L'encapsulation (getters / setters)
- L'héritage
- Le polymorphisme
- Les classes abstraites
- Les interfaces

1-Classes et objets : la base

Une **classe** est un modèle qui décrit la structure et le comportement d'un objet.

Un **objet** est une instance concrète de cette classe.

Analogie :

- ❖ La **classe** = le plan d'une maison donc L'**objet** = la maison construite
- ❖ La **classe** = un moule donc L'**objet** = la brique fabriquée avec ce moule

Exemple en PHP

Classe Personne

```
<?php
class Personne {
    public $nom;
    public $age;
```



```

// Constructeur
public function __construct($nom, $age) {
    $this->nom = $nom;
    $this->age = $age;
}

// Méthode
public function sePresenter() {
    echo " Bonjour, je m'appelle {$this->nom} et j'ai {$this->age} ans.<br>";
}
}

```

Création d'objets

```

$personne1 = new Personne("Chrislain", 27);$personne1->sePresenter();
$personne2 = new Personne("Amina", 22);$personne2->sePresenter();

```

À retenir:

Classe : modèle

Attributs : nom, age

Constructeur : initialise les valeurs

Méthode : sePresenter()

Exercice :

Créer une classe **Voiture** avec :

attributs : marque, annee

un constructeur

méthode afficherDetails()

créer un objet et tester

2-Encapsulation : protéger les données

L'encapsulation consiste à **cacher les données internes d'une classe** pour éviter leur modification directe.

En PHP, on utilise :

private : impossible d'accéder depuis l'extérieur

public : accessible partout

getters / setters pour contrôler l'accès

Exemple : Compte Bancaire

```
<?php
class CompteBancaire {
    private $solde = 0;

    // Getter
    public function getSolde() {
        return $this->solde;
    }

    // Setter avec validation
    public function setSolde($montant) {
        if ($montant >= 0) {
            $this->solde = $montant;
        } else {
            echo "    Le montant ne peut pas être négatif !<br>";
        }
    }

    // Méthode
    public function afficherSolde() {
        echo " Solde actuel : {$this->solde} FCFA<br>";
    }
}
```

Utilisation

```
$compte = new CompteBancaire();
$compte->setSolde(10000); $compte->afficherSolde();
$compte->setSolde(-500); // Erreur
```

3-Héritage : réutiliser le code

L'héritage permet à une classe d'hériter :

- des attributs
- des méthodes d'une classe parente.

Exemple : Animal → Chien / Chat

Classe parente

```
<?php
class Animal {
    public $nom;

    public function __construct($nom) {
        $this->nom = $nom;
    }

    public function parler() {
        echo "{$this->nom} fait un bruit.<br>";
    }
}
```

Classe Chien

```
class Chien extends Animal {
    public function parler() {
        echo " {$this->nom} aboie : Ouaf Ouaf !<br>";
    }
}
```

Classe Chat

```
class Chat extends Animal {
    public function parler() {
        echo " {$this->nom} miaule : Miaou !<br>";
    }
}
```

Exemple d'utilisation

```
$chien = new Chien("Rex");$chat = new Chat("Mina");
$chien->parler();$chat->parler();
```

4- Polymorphisme : même méthode, comportement différent

Le polymorphisme permet d'appeler la même méthode (parler()) sur différents objets, tout en exécutant leur version spécifique.

Exemple :

```

$animaux = [
    new Chien("Max"),
    new Chat("Luna"),
    new Animal("Créature")
];
foreach ($animaux as $animal) {
    $animal->parler(); // polymorphisme
}

```

5-Classe abstraite : un modèle incomplet

Une classe abstraite :

- ne peut pas être instanciée
- contient des méthodes que les classes dérivées doivent implémenter

Exemple : Véhicule abstrait

```

<?php
abstract class Vehicule {
    abstract public function demarrer();
    abstract public function arreter();
}

```

Classe Voiture

```

class Voiture extends Vehicule {
    public function demarrer() {
        echo " La voiture démarre !<br>";
    }

    public function arreter() {
        echo " La voiture s'arrête.<br>";
    }
}

```

Utilisation

```

$v = new Voiture();$v->demarrer();$v->arreter();

```

6- Interface : définir un contrat

Une interface contient uniquement des méthodes **sans implémentation**

Une classe qui implémente l'interface doit définir toutes les méthodes.

Exemple : Interface Paiement

```
<?php
interface Paiement {
    public function payer($montant);
}
```

Paiement mobile

```
class PaiementMobile implements Paiement {
    public function payer($montant) {
        echo " Paiement mobile de $montant FCFA via FlutterPay !<br>";
    }
}
```

Paiement par carte

```
class PaiementCarte implements Paiement {
    public function payer($montant) {
        echo " Paiement par carte de $montant FCFA réussi !<br>";
    }
}
```

Exemple d'utilisation

```
$p1 = new PaiementMobile();$p2 = new PaiementCarte();
$p1->payer(5000);$p2->payer(12000);
```